

# The Application Layer

Where the Internet became something people needed

**Yaxiong Xie**

CSE589 · Modern Network Concepts · Fall 2026



BEFORE WE DEFINE ANYTHING

# 6 billion

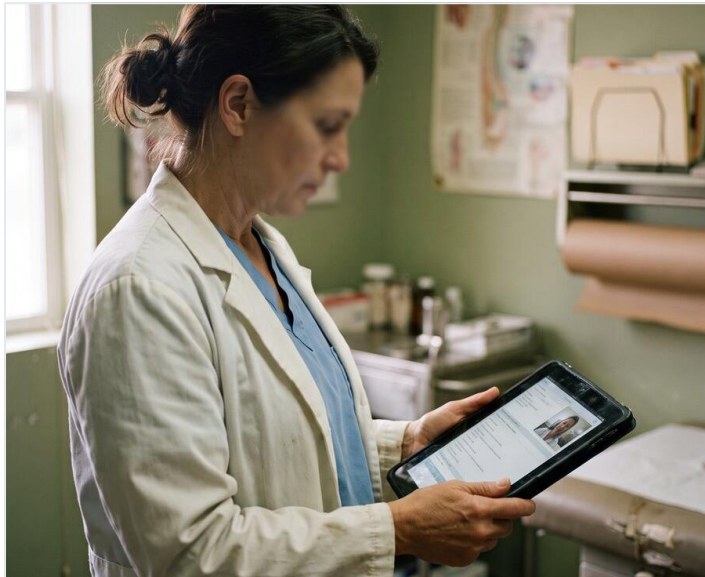
people used the Internet in 2025 — about three in every four people alive.

---

Almost none of what they did with it existed when this network was designed.

**So who added all of it — and what did the network give them to work with?**

# None of this needed a new network



## A scan read 400 km away

The clinic did not get better equipment. It got a second opinion.



## A stall that never needed a bank

Payment became software running on two ordinary phones.

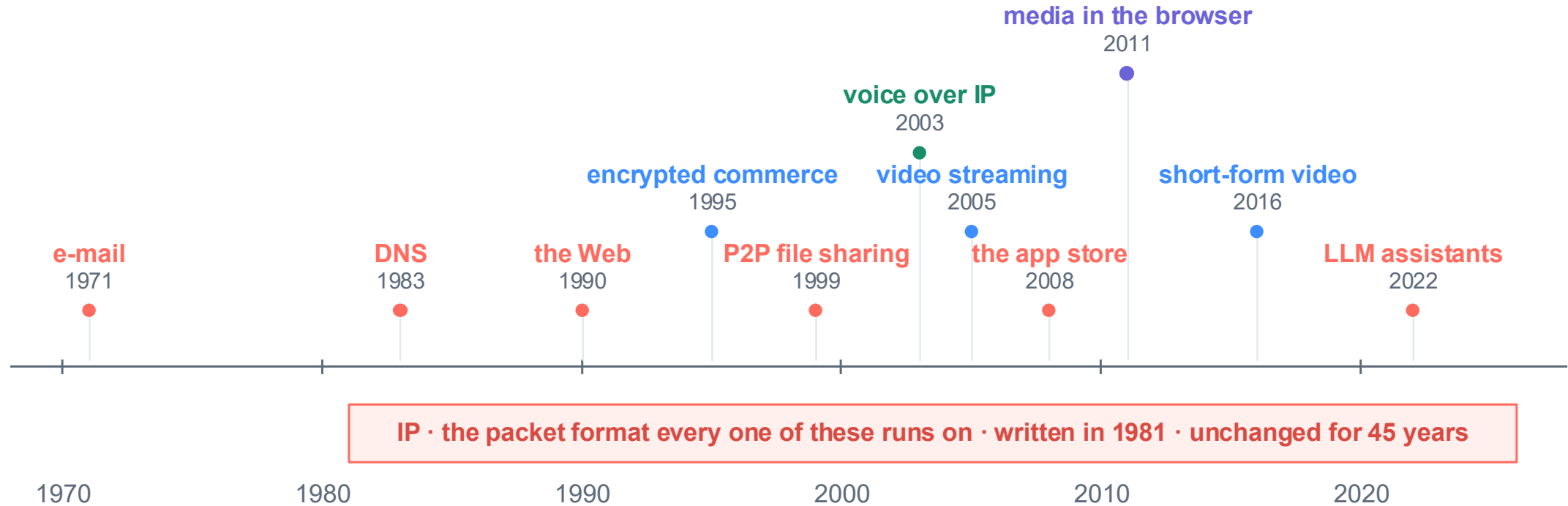


## A classroom where the student is

The lesson is not in the building any more. It is in the protocol.

**The wire did not change. What people could do with it did.**

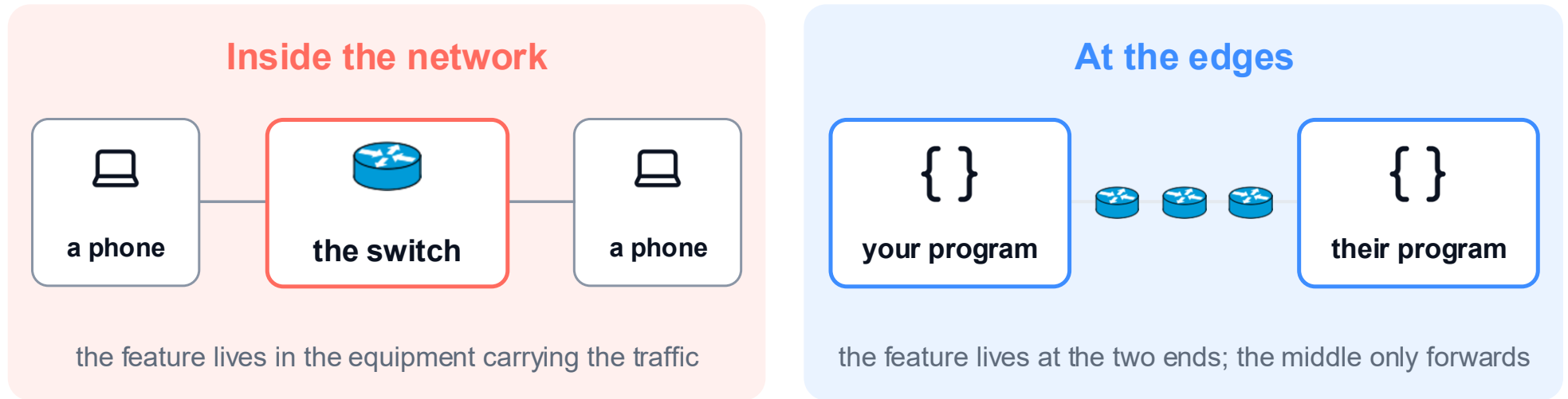
# One packet format. Fifty years of applications.



**Nobody upgraded a router for any of these. Why was that even possible?**

IPv6 changed how long an address is. It did not change the idea: the core forwards, the edges interpret.

# Because the intelligence was never put in the middle



|              |                                      |                                     |
|--------------|--------------------------------------|-------------------------------------|
| New feature  | new software in every switch         | two programs, on two ordinary hosts |
| Approval     | whoever owns the switching equipment | nobody                              |
| Time to ship | years                                | as long as it takes you to write it |

A network that understands your application can also refuse it.

- So what did you actually get?



# An address that works, and a payload nobody reads



## 1 · You get an address that works

Hand the network a destination and the packet goes there. No setup call, no operator told in advance, nobody's permission.

### DESTINATION

128.205.32.10

### SOURCE

203.0.113.7

### PROTO

UDP

### PAYLOAD

20 ms of Alice's voice

## 2 · You get a payload nobody reads

Whatever you put in here is carried untouched. Not one device on the path has an opinion about it — or the ability to act on it.

### DESTINATION

128.205.32.10

### SOURCE

203.0.113.7

### PROTO

UDP

### PAYLOAD

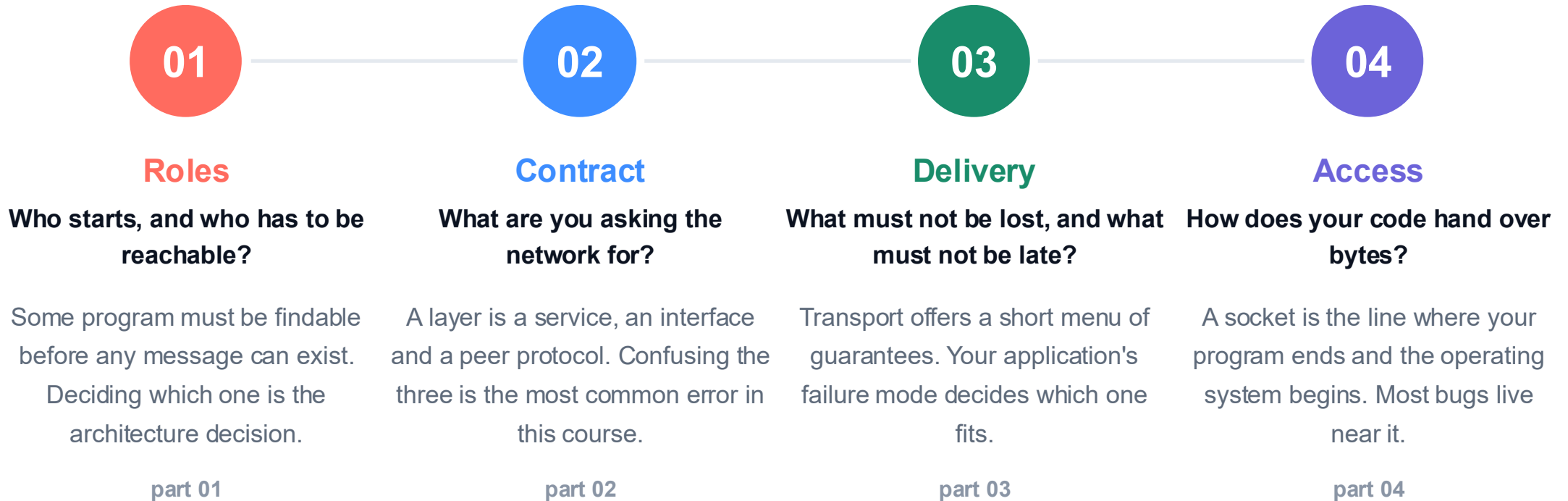
20 ms of Alice's voice

**That is the entire list.**

Not who is reachable · not what these bytes mean · not whether they arrive in time · not how your code hands them over

**Those four leftovers are your job—and the next four parts of this lecture.**

# You are about to write one. Four questions you must answer.



**By the end you should be able to take any application apart along these four lines.**

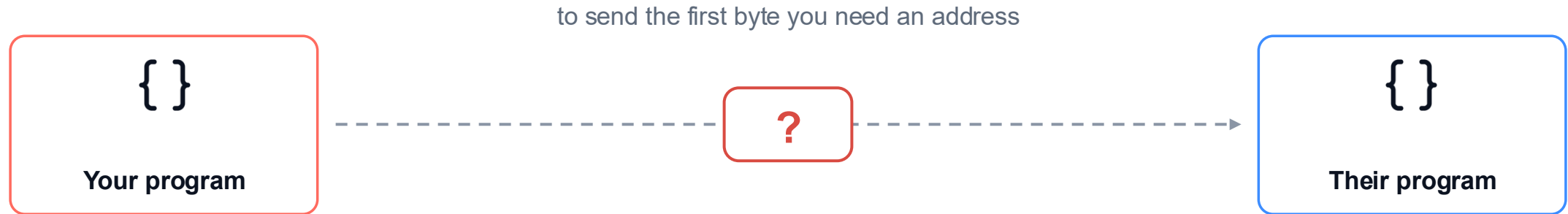
# Before anyone can talk, who has to be there?

## 01

### Application architecture

Architecture is not about topology diagrams. It is the answer to one question: which program is guaranteed to be reachable before the conversation exists?

# You cannot send a message to someone you cannot find



## Addresses move

A laptop is given a different IP address on every network it joins. A phone changes address walking down the street. Yesterday's address is not a way to reach anyone today.

## Most hosts cannot be called

Home and campus networks put you behind a NAT or a firewall, which discards connections arriving from outside. Your machine can dial out; almost nothing can dial in.

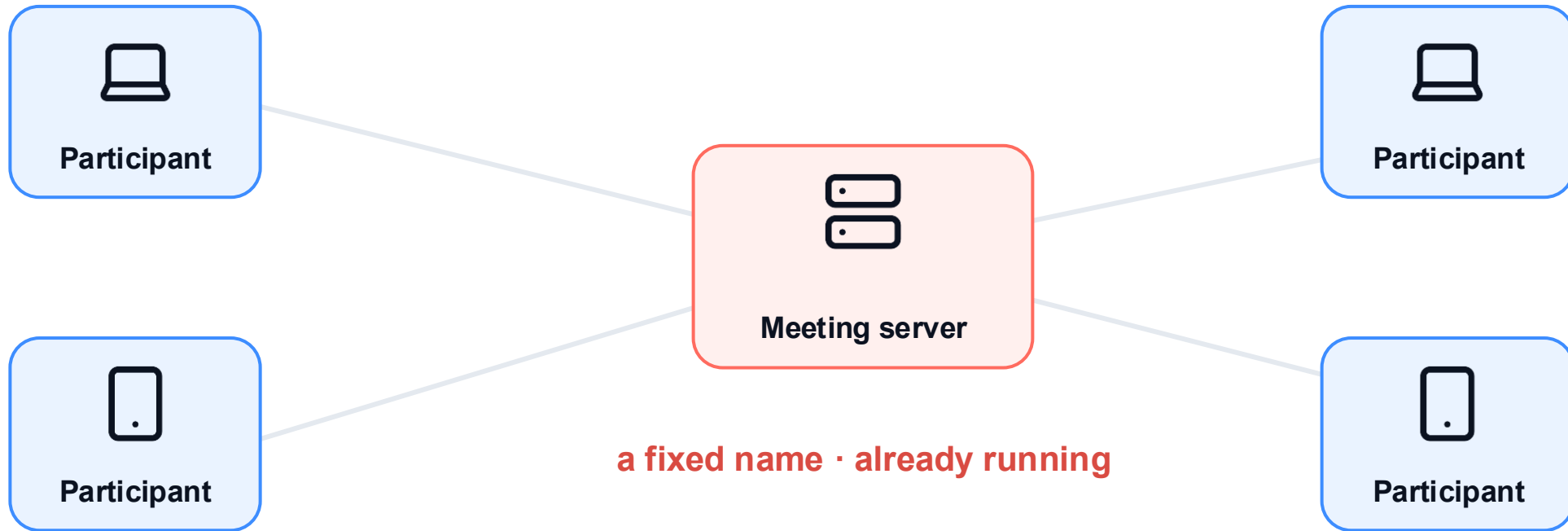
## Programs are not always running

The other side may be closed, asleep, or never installed. An address is useless if no process is listening at it when your packet arrives.

**Every architecture is one answer to: which side is guaranteed to be there?**

# Client-server: make one side permanently findable

Give up on finding everybody. Elect one program to be reachable, and let every conversation start there.

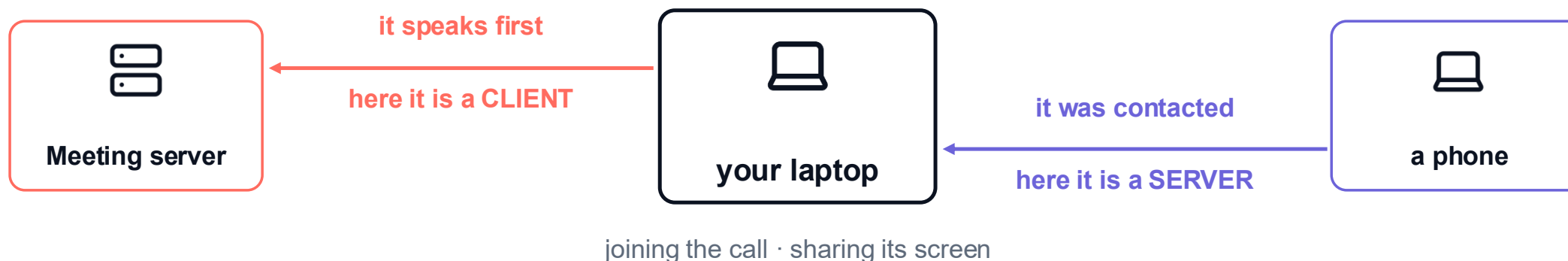


**The client must know the server. The server never has to know the client.**

- A misconception worth killing early

# Client and server are roles, not kinds of machine

One laptop. One moment in time. Both roles at once.



## CLIENT — the process that speaks first

It must already know where to send. It needs no address that anyone else knows in advance, and it can sit behind a NAT.

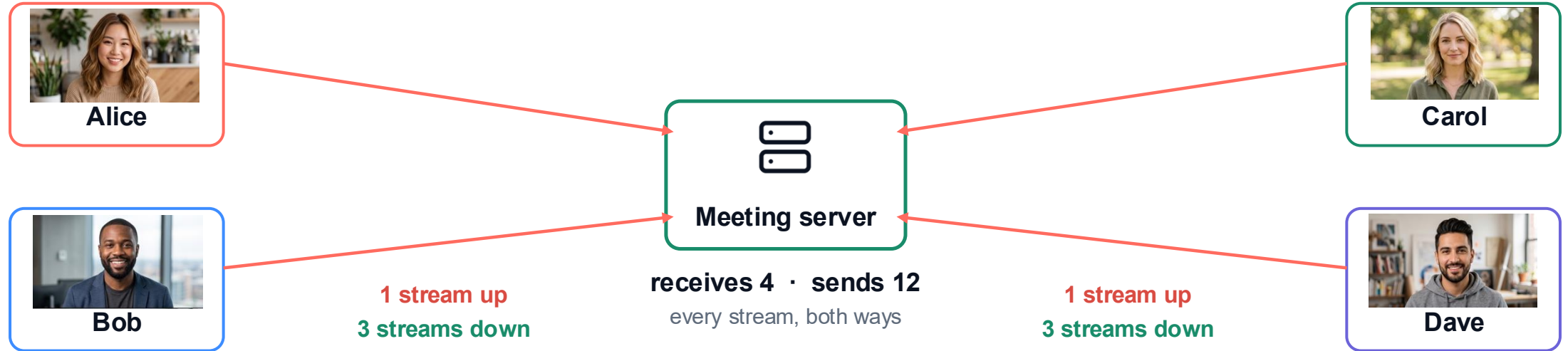
## SERVER — the process that waits

It starts before anyone needs it, binds an address others already know, and then does nothing at all until it is contacted.

**Roles are assigned per conversation. The same program swaps them minute to minute.**

# Everything goes to the server, and comes back out again

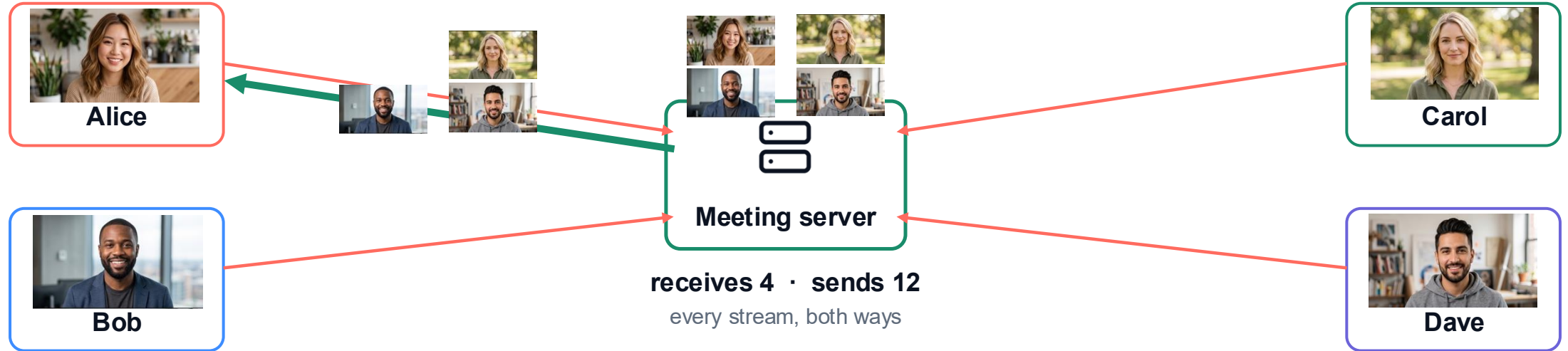
A four-person call. Each camera produces one stream; each person has to see the other three.



- The cost of the always-on side

# Everything goes to the server, and comes back out again

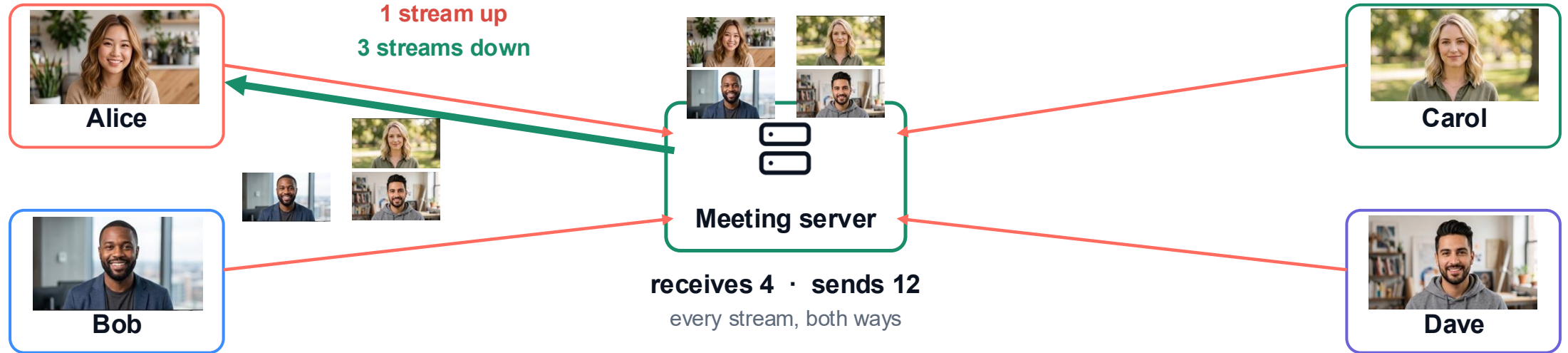
A four-person call. Each camera produces one stream; each person has to see the other three.



- The cost of the always-on side

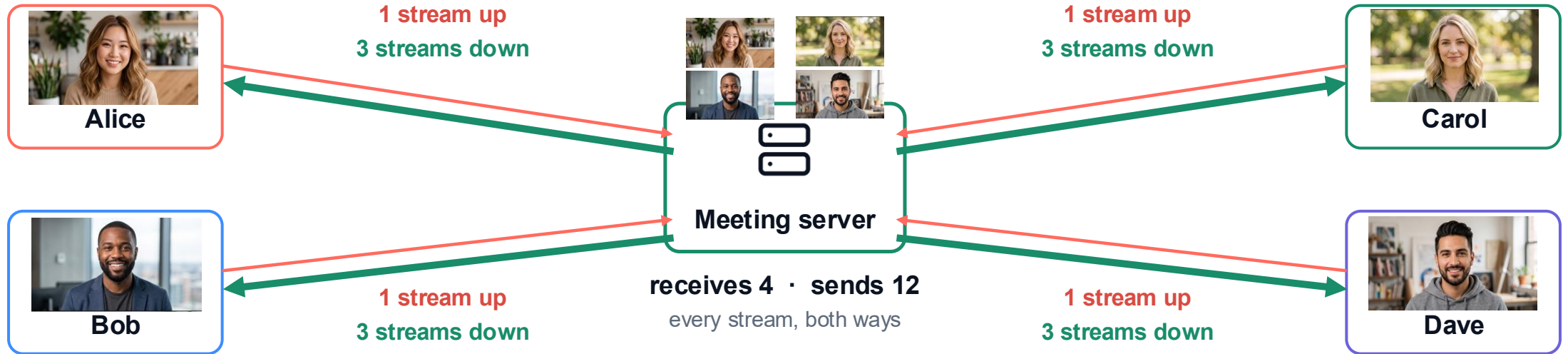
# Everything goes to the server, and comes back out again

A four-person call. Each camera produces one stream; each person has to see the other three.



# Everything goes to the server, and comes back out again

A four-person call. Each camera produces one stream; each person has to see the other three.

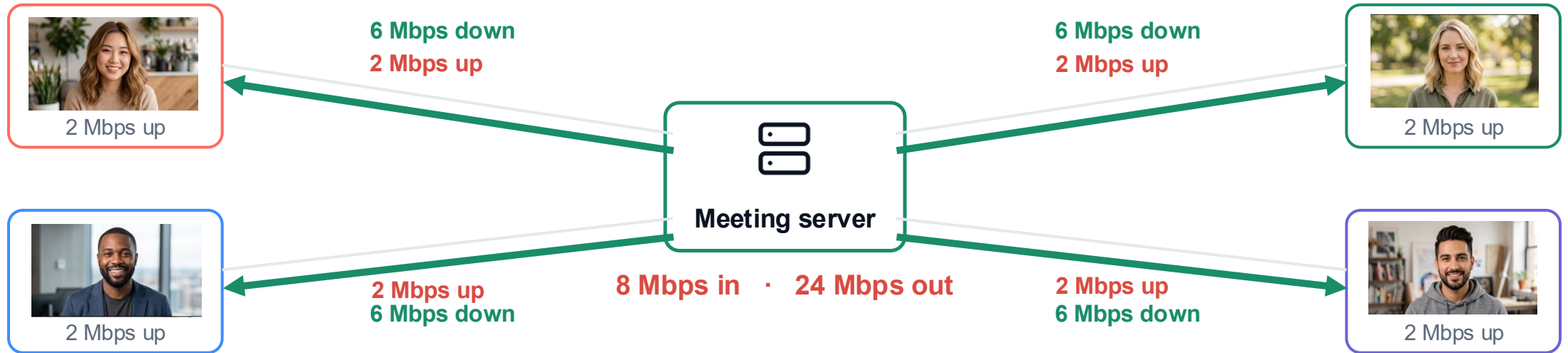


Nobody sends anything to anybody else. Every frame Alice's camera produces is uploaded to the server, and the server then sends a copy of it to Bob, to Carol and to Dave.

- The cost of the always-on side

# Every one of those streams is on somebody's bill

Same picture, now with the meter running. Take each camera at 2 Mbps.



## The operator pays for the out arrows

24 Mbps sustained is about 10 GB for a one-hour call. Cloud egress is billed per gigabyte, so the cost of the call scales with how many people are in it.

## And pays for the idle capacity

The server has to be big enough for the busiest minute of the week, and is sitting there — paid for — during every quiet minute as well.

**The out arrows are three times the in arrows—and they are the ones you pay for.**

# One box everyone depends on can stop everyone

Nothing has changed on any of the four laptops. The call is over anyway.



## Every path went through it

There was no route from Alice to Bob that did not pass through the server, so there is nothing for the four laptops to fall back to.

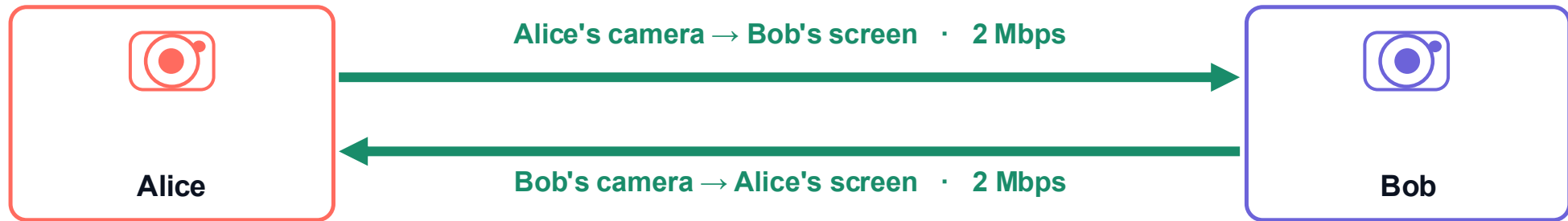
## And they are all still fine

Four working computers, four working cameras, four working network connections — and no call. The failure is entirely in the architecture.

**A single always-on address concentrates the service, the bill and the risk.**

# Peer-to-peer: the video goes straight to the other user

Same two people, same cameras, same 2 Mbps each.



## Every new person brings their own uplink

A server's capacity is consumed by each extra user. Here each extra user arrives carrying the bandwidth needed to serve their own video.

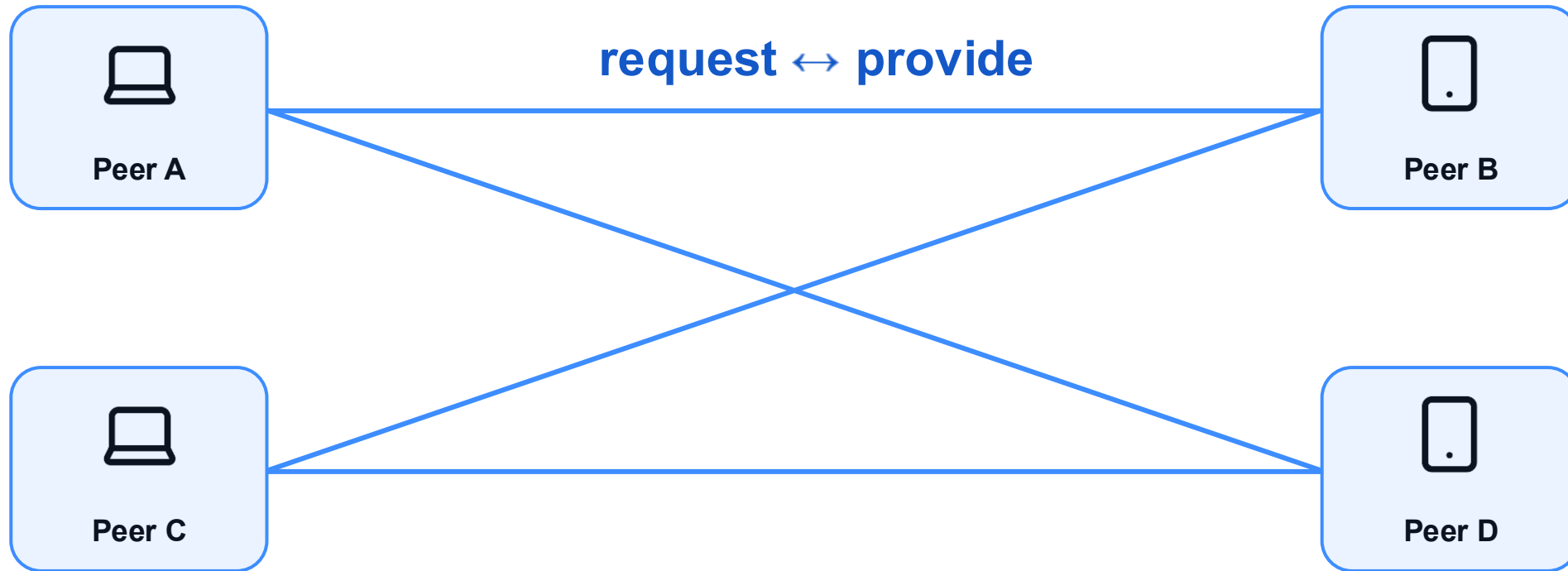
## And no single box can end the call

The frames never touch a machine either of them pays for. If the operator's server goes down mid-call, the media keeps flowing.

**A new user adds load to a server, but adds a supplier to a peer network.**

# Peer-to-peer: let every participant supply capacity too

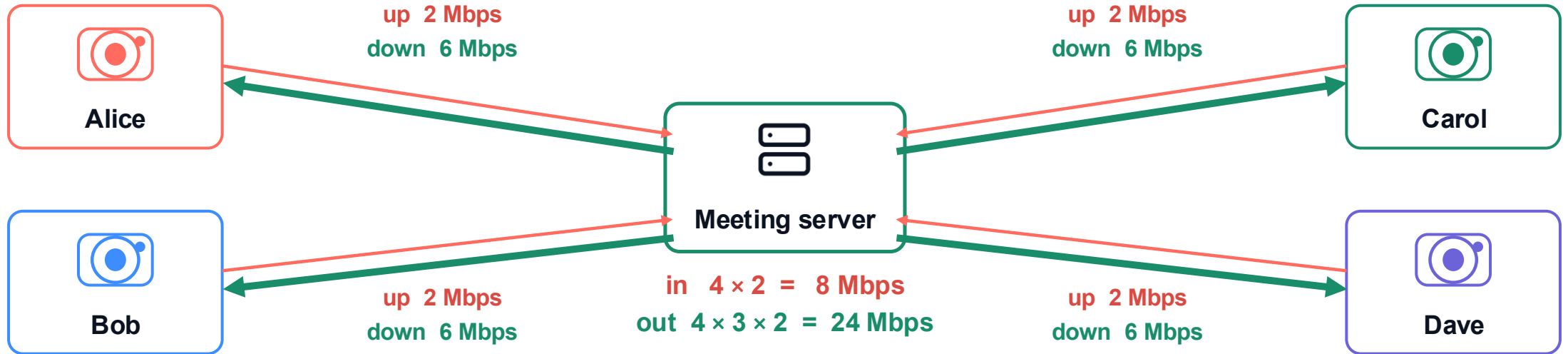
If each side can both request and provide, the system gains capacity as it gains users instead of spending capacity on them.



**A new user adds load to a server, but adds a supplier to a peer network.**

# Cost, worked out: everything through a server

$N = 4$  people. Every camera sends 2 Mbps. Everyone must see everyone else.



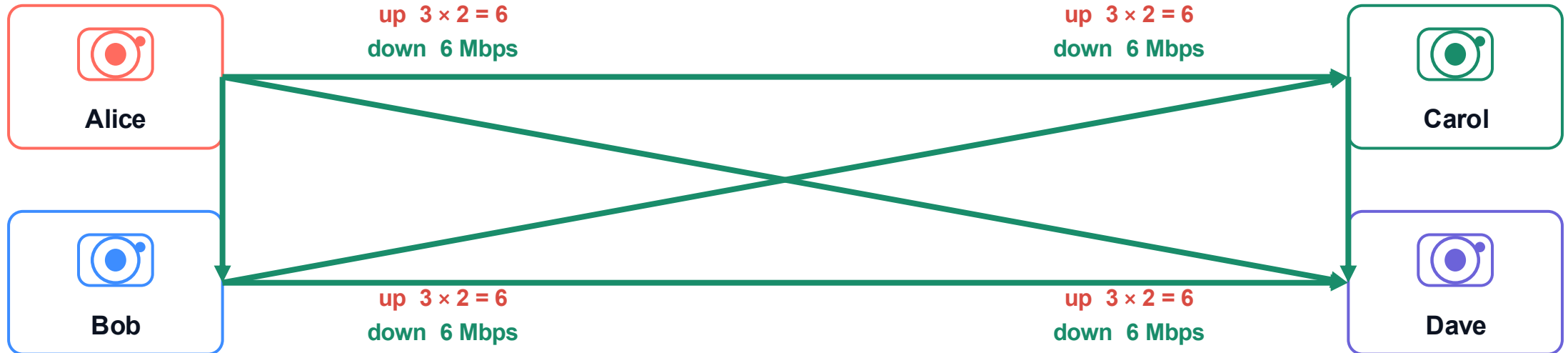
## What the arithmetic says

Each laptop uploads one stream and downloads  $N-1$ . The server uploads  $N \times (N-1)$  streams — so its bill grows with the square of the number of people, while nobody else's changes at all.

**One upload each—and the server sends a copy of every one of them to everybody else.**

# Cost, worked out: every peer sends to every peer

Same four people, same 2 Mbps cameras, no server on the media path — six links, each carrying two streams.



## What the arithmetic says

The server uploads nothing. Each laptop now uploads  $N-1$  streams instead of one — so the cost has not disappeared, it has been moved onto the participants, and it grows with  $N$  on every one of them.

**Nothing was saved. The upload just moved off the server and onto every laptop.**

## Side by side—then set $N = 40$

| $N = 4$               | Through a server                | Full mesh                     |
|-----------------------|---------------------------------|-------------------------------|
| each laptop uploads   | $1 \times 2 = 2 \text{ Mbps}$   | $3 \times 2 = 6 \text{ Mbps}$ |
| each laptop downloads | 6 Mbps                          | 6 Mbps                        |
| the server uploads    | $12 \times 2 = 24 \text{ Mbps}$ | nothing                       |
| grows with            | $N^2$ on the server             | $N$ on every laptop           |

Now  $N = 40$  — what each laptop must upload in a full mesh



Neither shape survives  $N = 40$ . That is why real systems are neither.

# P2P does not remove the hard problems—it moves them

## What the server was quietly doing

- **Being findable**

It had one name that resolved to one place, so a client could send the first packet knowing nothing else about the world.

- **Knowing who exists**

It held the authoritative list of participants and noticed within seconds when one of them vanished.

- **Being already trusted**

There was a single copy of the room's state, held by a party everyone had already authenticated to.

## What you must now build yourself

- **Discovery**

Some way for a peer to learn another peer's current address: a tracker, a distributed hash table, or — in practice — a server.

- **Churn**

Peers disappear without warning. Whatever state they held has to be re-created or re-fetched from whoever is still online.

- **NAT traversal**

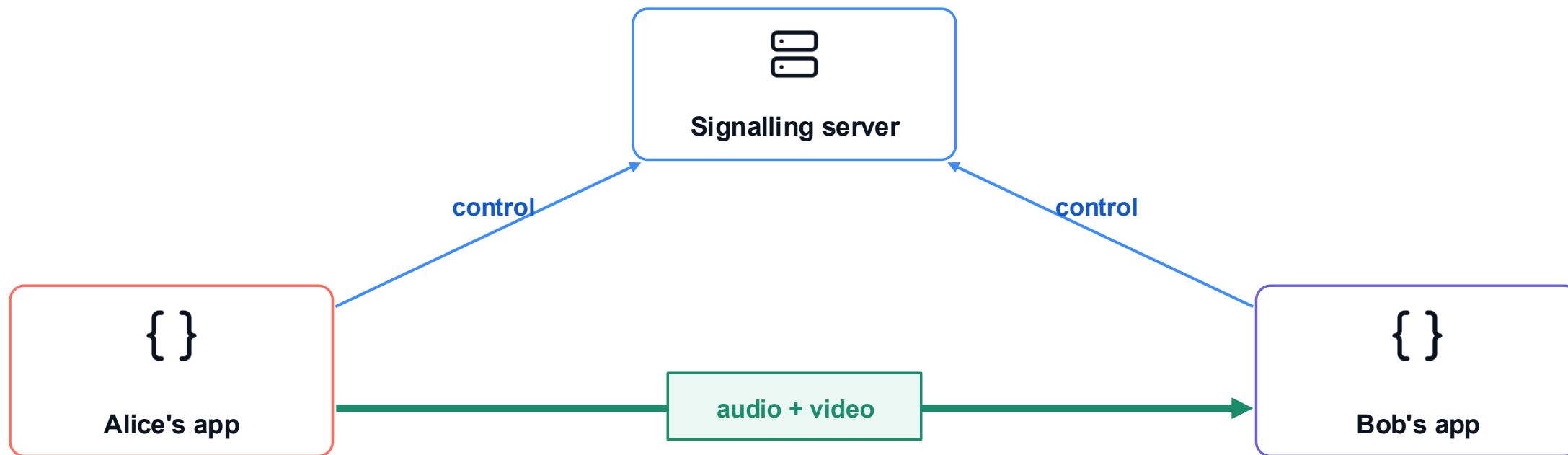
Two peers behind NATs cannot simply connect to each other. ICE, STUN and TURN are three entire protocols that exist because of this one problem.

**Every deployed P2P system has centralised parts. They are just not on the data path.**

**Removing the server does not remove the problems the server was solving.**

# One call, two completely different flows

The same application runs both at the same time. They disagree about size, rate, and what counts as failure.



## Control path · via the server

who is in the room · identity · encryption keys · “Bob joined”  
a few kilobytes in total · a second late is fine · must be right

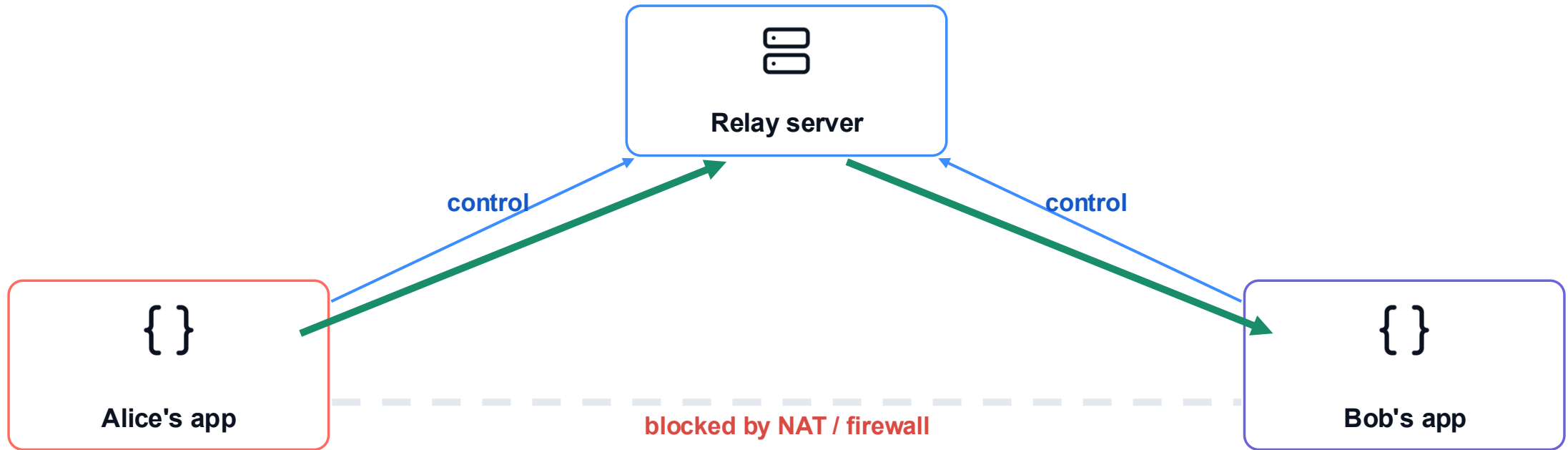
## Data path · directly between peers

the audio and video frames themselves  
1–3 Mbps each way, continuously · 50 frames a second · 150 ms  
late is worthless

**Small, rare and authoritative goes to a server. Large, continuous and urgent goes direct.**

# When the direct path is blocked, media goes via a server

Same application, same call, one change: the direct path did not come up, so the frames take the long way round.



Nothing about the application changed. The same code, the same protocol and the same server are still in use — but the operator is now paying for every media byte instead of a few kilobytes of signalling, and every frame takes a longer path.

**Architecture is chosen per data path, at run time—and it can change during one call.**

# Choosing an architecture is choosing who pays for what

## Reach

**Who must be findable before anything happens?**

A server has a permanent name; a peer usually has neither a stable address nor a way to accept incoming connections.

## Capacity

**Whose upstream bandwidth carries the traffic?**

A server's cost grows with the number of users. A peer network's capacity grows with them instead — until each peer's own uplink runs out.

## Failure

**What stops working when one participant disappears?**

The server is a single point of failure for everyone. A peer leaving usually degrades the system rather than stopping it.

## Trust

**Who is authoritative about shared state?**

One party everyone authenticated to, or a state that must be negotiated and reconciled among mutually suspicious peers.

**There is no free option—only a choice about which cost you would rather carry.**

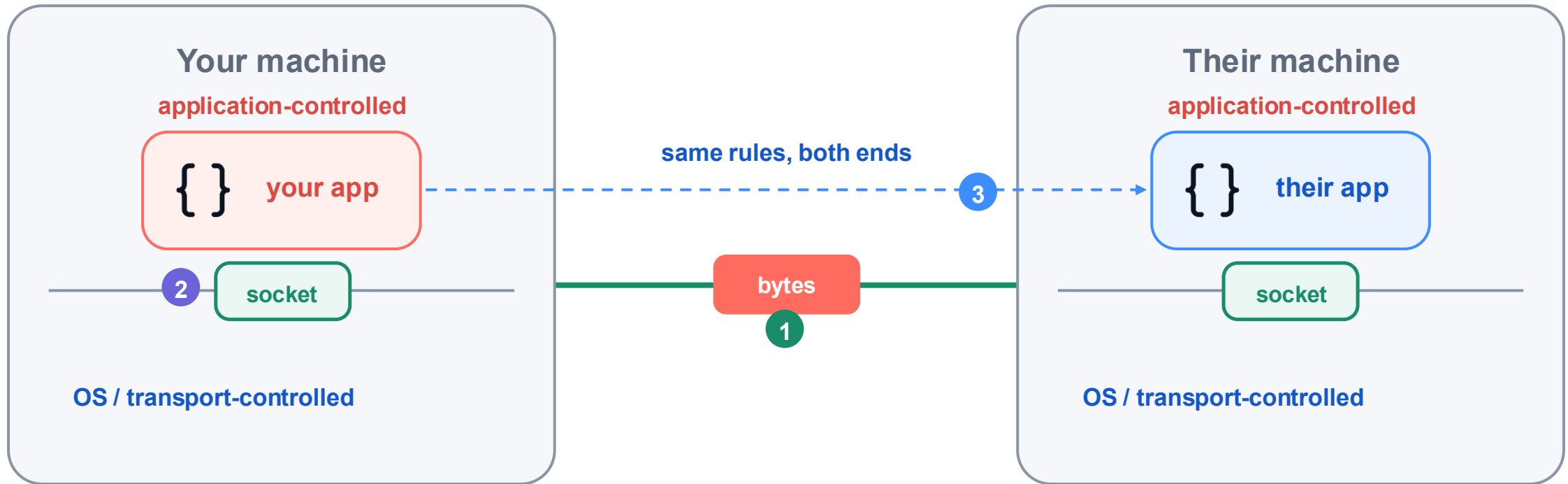
# What are you actually asking the network for?

## 02

### **Service · interface · protocol**

One layer, described three different ways. Nearly every confusion in this course comes from swapping one of them for another.

# Three questions, three directions on one picture



## 1 SERVICE — what you get

What the layer below promises — a reliable byte stream, or bare datagrams.

## 2 INTERFACE — how you ask

The calls your program makes to reach it. Local only: the peer never sees them.

## 3 PROTOCOL — what you agree on

The rules both application processes follow so the bytes mean something.

**Service is vertical. Interface is local. Protocol is horizontal.**

# Service is what you get. Interface is how you ask.

## Service — the promise

- **Stated as a guarantee, not as code**

“Every byte you hand me arrives, once, in order.” That sentence is the whole of TCP's service, and it names no function at all.

- **Identical in every language**

A Python program and a C program get the same guarantee, because the guarantee belongs to the transport layer, not to the API.

- **You pick it, then live with it**

Choosing TCP or UDP is choosing a contract. Everything above has to be designed around what that contract does not cover.

## Interface — the request

- **A set of purely local calls**

socket(), bind(), connect(), send(), recv(), close(). They happen on your machine; nothing about them crosses the network.

- **Replaceable without consequence**

Berkeley sockets, Winsock, Java NIO and Go's net package expose the same service through completely different code.

- **It hides mechanism, not the contract**

The API will never tell you a segment was retransmitted — and it can never hand you a guarantee the service does not have.

**Rewrite your networking code in another language: the interface changes, not one guarantee.**

**If you cannot state the service in one sentence without naming a function, you do not know it yet.**

# A protocol is the only part you must agree on with a stranger

Two programs with nothing whatever in common, except one thing.



## Private — unobservable from the other side

Language, structure, storage, libraries, threading. Two implementations of one protocol usually share no source code at all.

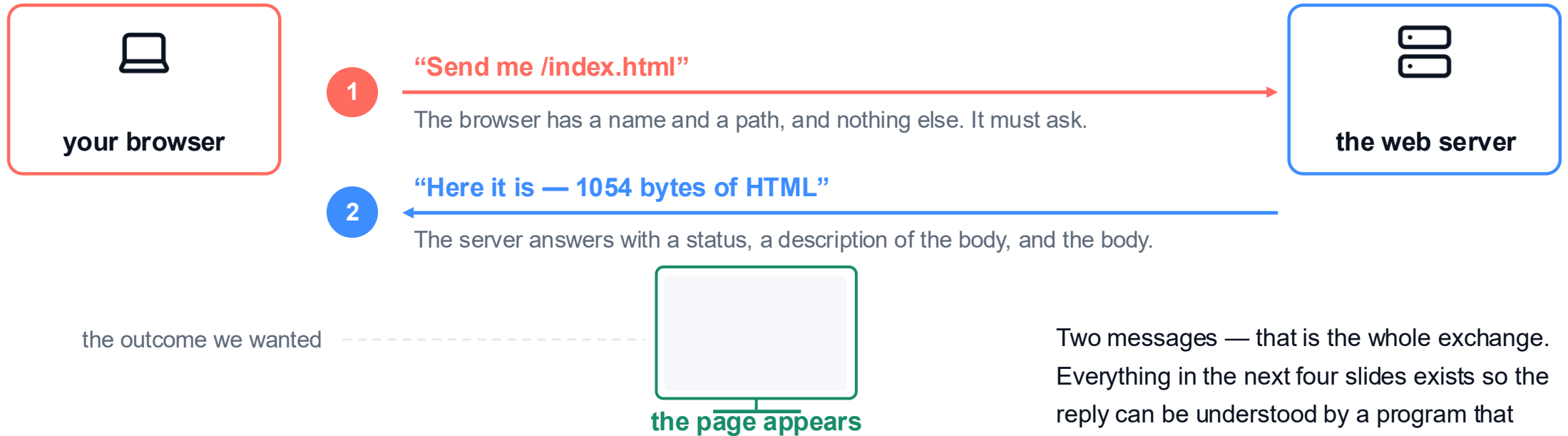
## Public — must match exactly

Field order, lengths, meanings, and which message may legally follow which. Change any of it and every peer in the world breaks.

**Your browser and a web server, written by strangers decades apart, still interoperate.**

# What we are trying to achieve: one page on the screen

Before any protocol detail — here is the goal, and the two messages it takes.



# Type — which messages are allowed to exist

The receiver's very first decision: what kind of message is this?

the client sends

```
GET /index.html HTTP/1.1
```

```
Host: www.buffalo.edu
```

```
Accept: text/html
```

the server replies

```
HTTP/1.1 200 OK
```

```
Content-Type: text/html
```

```
Content-Length: 1054
```

```
<html> ... 1054 bytes ...
```

## Named in the first line, and nowhere else

GET is a request type; 200 OK is a response type. A receiver branches on this before it looks at anything else, because everything else means something different depending on it.

## A message with no defined type cannot be handled

If your protocol lets a peer send something the specification does not list, the receiver has no rule to apply and no defined way to fail. That is a protocol bug, not an implementation bug.

**Fix the list of message types first. Everything else is defined per type.**

# Syntax — how a message is laid out in bytes

Given the type, where exactly is each piece of information?

the client sends

|                          |
|--------------------------|
| GET /index.html HTTP/1.1 |
| Host: www.buffalo.edu    |
| Accept: text/html        |

the server replies

|                         |
|-------------------------|
| HTTP/1.1 200 OK         |
| Content-Type: text/html |
| Content-Length: 1054    |

<html> ... 1054 bytes ...

## One line each, name colon value, and a fixed order for the first line

The method, the target and the version are positional and separated by single spaces. Headers are order-independent but each is a name, a colon and a value. This is precisely what a parser implements.

## A blank line ends the header section

Not a convention — a rule. It is how the receiver knows the headers have finished and the body, if any, begins on the next byte.

**Syntax is what a parser can check. It says nothing yet about meaning.**

# Semantics — what each value actually claims

The bytes parsed cleanly. What are they actually claiming?

the client sends

```
GET /index.html HTTP/1.1
```

```
Host: www.buffalo.edu
```

```
Accept: text/html
```

the server replies

```
HTTP/1.1 200 OK
```

```
Content-Type: text/html
```

```
Content-Length: 1054
```

```
<html> ... 1054 bytes ...
```

**200 means the resource is here; 404 would mean it is absent**

Not “the server is broken” and not “try again”. Each status commits the sender to a specific claim about the world, and the receiver is entitled to act on it. Meaning is agreed in advance and cannot be recovered from the bytes.

**GET commits the client to asking for nothing but a copy**

A server may therefore cache it, retry it, or serve it from a proxy. Those optimisations are only legal because the meaning of GET was agreed.

**Semantics is the part you cannot test with a parser—only against the specification.**

# Rules — who may send what, and when

Both sides understand every message. In what order are they allowed to arrive?

the client sends

```
GET /index.html HTTP/1.1
```

```
Host: www.buffalo.edu
```

```
Accept: text/html
```

the server replies

```
HTTP/1.1 200 OK
```

```
Content-Type: text/html
```

```
Content-Length: 1054
```

```
<html> ... 1054 bytes ...
```

## A response follows a request, and never precedes one

Each side keeps state: the client is waiting, the server is serving. The legal orderings are as much a part of the protocol as the field layout, and a message arriving out of turn must have a defined outcome.

## Content-Length says where this message stops

Which is how the next one can begin. Without it the receiver cannot know whether the stream has ended or the body is still coming — the same framing problem we hit again in part 03.

**Four elements. Drop any one and the receiver is holding bytes it cannot act on.**

# Different jobs, different protocols—the same three questions

## Web

**Fetch a named resource**

HTTP, over TCP or QUIC. Request and response, one resource at a time, and no memory of you between requests unless something above it adds one.

## Mail

**Hand a message to a server that keeps trying**

SMTP to send, IMAP to read. Store-and-forward: minutes of delay are normal, and a server being down only postpones delivery.

## Real-time media

**Carry timed frames that expire**

RTP over UDP for the frames themselves, with SIP or WebRTC signalling to set the session up. Arriving late is worse than not arriving.

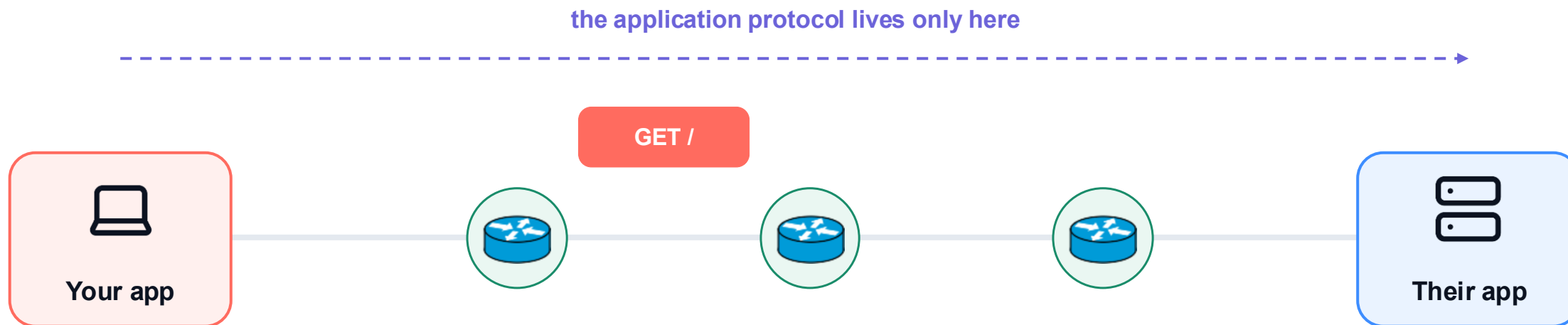
## Naming

**Turn a name into an address**

DNS: usually one small query out and one reply back. Almost everything else on this list depends on it completing first.

**We take these apart one at a time later. The layer model is what makes them comparable.**

# Only the two endpoints run the application protocol



These three routers have never heard of HTTP and do not need to. The dashed line is not a wire. It is an agreement between two programs, carried by a path that does not understand it.

**Which is the whole reason you could invent a new protocol this afternoon.**

# What can you actually ask the network for?

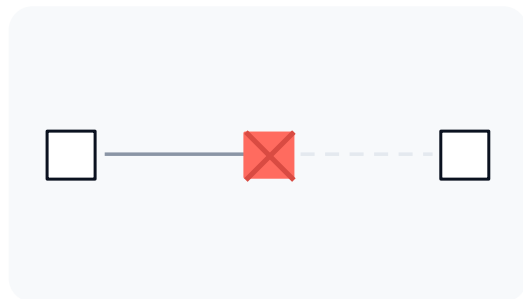
## 03

### The transport service

The menu is far shorter than students expect. Understanding why it is short is more useful than memorising what is on it.

# Four ways a delivery can go wrong

Before choosing a transport, decide which of these your application cannot survive.



## Loss

**Can any of it simply be missing?**

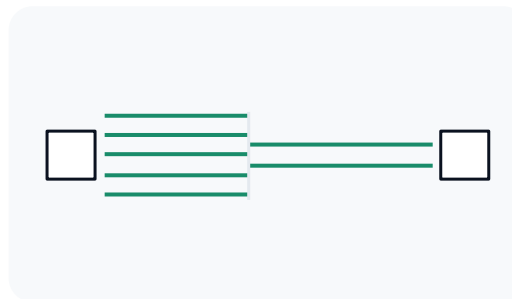
A missing byte in a bank transfer is a disaster. A missing 20 ms of audio is a click nobody notices.



## Delay

**When is late the same as never?**

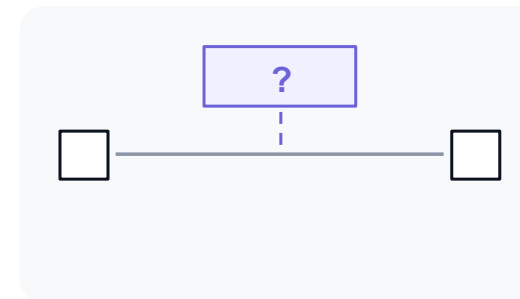
A voice frame has about 150 ms before the moment it should have been played. An e-mail has hours.



## Rate

**How much per second does it need?**

Video quality follows the available rate continuously. A chat message needs a few hundred bytes and does not care.



## Exposure

**Who must not read or change it?**

Confidentiality, integrity, and knowing who the peer really is — none of it provided by the network.

**State the requirement first. Only then does the transport choice mean anything.**

# One application, four flows, four different answers

Everything below is happening inside a single video call, at the same moment.

| FLOW           | LOSS          | DELAY            | RATE                | WHAT IT IMPLIES             |
|----------------|---------------|------------------|---------------------|-----------------------------|
| • Voice        | tolerable     | under ~150 ms    | ~50 kbps            | late is worse than lost     |
| • Video        | tolerable     | under ~150 ms    | 1–3 Mbps            | drop quality, never stall   |
| • Chat message | not tolerable | seconds are fine | a few hundred bytes | every character must arrive |
| • Shared file  | not tolerable | minutes are fine | as fast as offered  | correctness, then speed     |

Two of these rows want loss repaired and will wait for it. Two want the data now and will accept gaps. No single contract satisfies both.

**“Which transport does this app use?” is the wrong question. Ask it per flow.**

# The menu has two items

## TCP

an ordered stream of bytes

you send



carried by TCP

they receive



one of them was lost and re-sent — you never found out

The OS repairs loss, hides duplicates and reordering, and decides how fast you may send. You read a clean stream and think about nothing else.

## UDP

independent datagrams

you send



carried by UDP

they receive



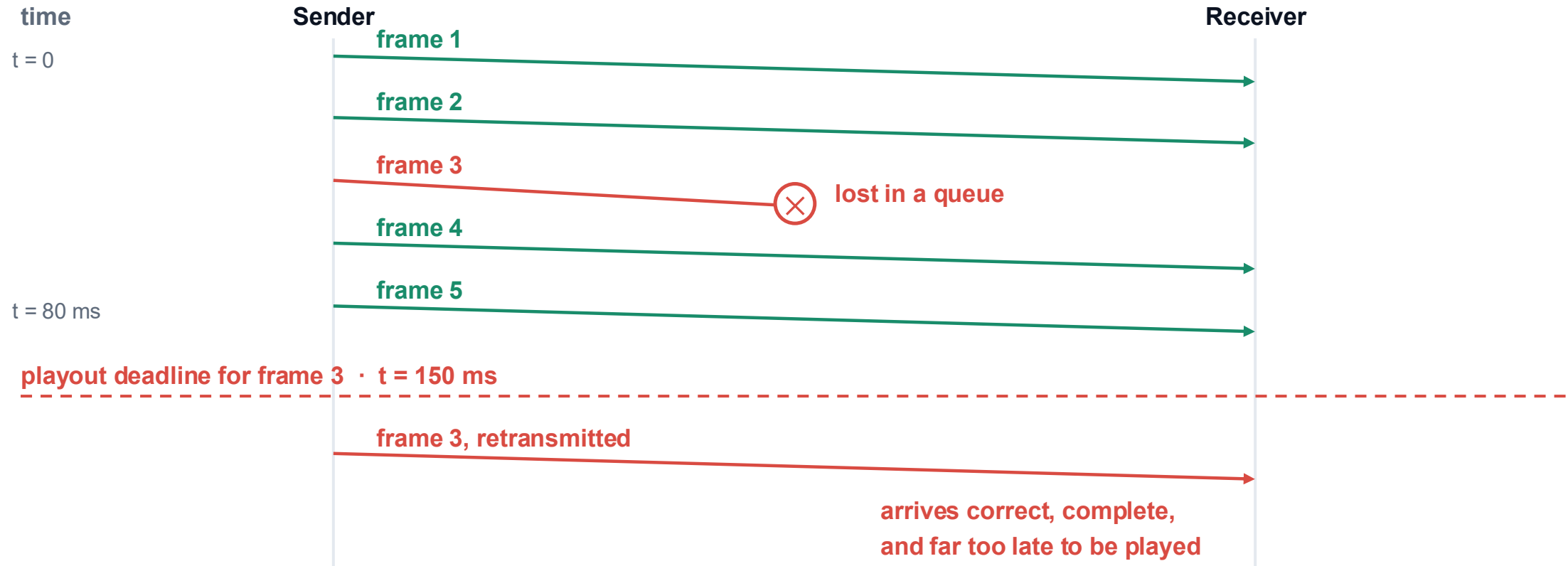
3 was lost and 5 overtook 4 — you see exactly that

Nothing is repaired, re-ordered or paced. Every one of those decisions is still yours to make — which is the point.

**Neither is better. They divide the same work between the OS and you, differently.**

# Why “reliable” is the wrong choice for a voice call

A 20 ms voice frame every 20 ms. One is lost. Watch what TCP does next.



Worse: frames 4 and 5 arrived on time, and TCP will not hand them to the application until frame 3 is repaired. One lost frame silences three.

# Every TCP guarantee is paid for with something

## Reliable

**Nothing you send is lost**

Loss is repaired by sending it again — so the repaired data arrives at least one round trip later than everything around it.

## Ordered

**Bytes arrive in the order you sent them**

So one missing segment holds back everything received behind it. The receiver has your data and is not allowed to give it to you yet.

## Controlled

**You will not overrun the peer or the path**

Your sending rate is set by feedback loops you do not control and cannot observe through the socket API.

## Connected

**Both sides agree before data moves**

A handshake before the first byte — clearly visible on a short exchange across a long path.

**TCP's method for handling loss is delay. For live media that is the wrong trade.**

# What UDP refuses to do—and why you might want that

## No resend

**A lost datagram stays lost**

Exactly right for audio: a frame resent 200 ms later has no slot left to play in. Better to conceal the gap and keep going.

## No ordering

**Datagram 5 may arrive before 4**

A decoder can often use what it has immediately. Waiting for 4 would stall the picture to fix something the viewer would never have seen.

## No rate control

**You send at whatever rate you choose**

So a codec can lower its own quality in response to loss — a decision the application understands and the transport layer cannot.

## No handshake

**The first datagram is the first message**

One packet out and one back is the entire exchange for a DNS query. A connection setup would more than double its cost.

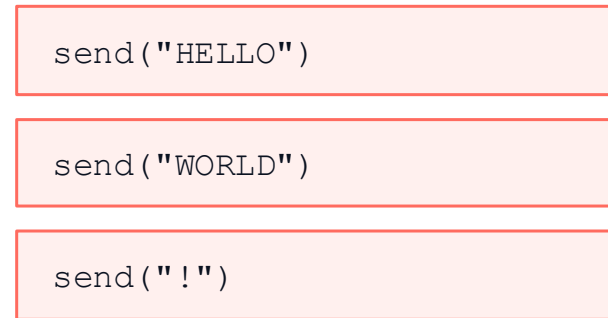
**UDP is not TCP with the features removed. It is transport with the decisions left to you.**

- A misconception with real consequences

# TCP gives you a stream of bytes—not your messages

The same three calls, seen from each end of one connection.

your program calls

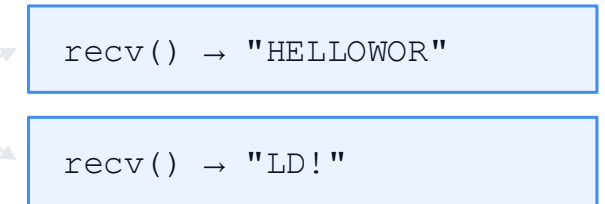


what actually travels



the boundaries you meant are not sent

their program receives



Three sends became two receives, split in a place your program never chose. Nothing is lost and nothing is out of order — TCP kept every promise it made.

If your protocol needs message boundaries, the protocol has to carry them: a length prefix, or a delimiter — which is exactly what HTTP's blank line and Content-Length were doing two sections ago.

**Framing is an application-layer job—and the most common bug in a first socket program.**

# Reliable delivery is not a reliable application

## What TCP promises

- **Every byte you handed in**

Delivered in order, without duplication, with loss repaired underneath without you seeing it.

- **Into the peer's receive buffer**

The kernel on the other machine has your data and has acknowledged it. That is the event TCP is talking about.

- **Or the connection fails**

If it cannot deliver, it eventually reports the connection as broken. Those are the only two outcomes it has.

## What it does not promise

- **That the peer read it**

The bytes can sit in that buffer while the application is blocked, busy, restarting, or has crashed since.

- **That the peer acted on it**

Acknowledgement is a transport event. Whether the order was actually placed is an application question with an application answer.

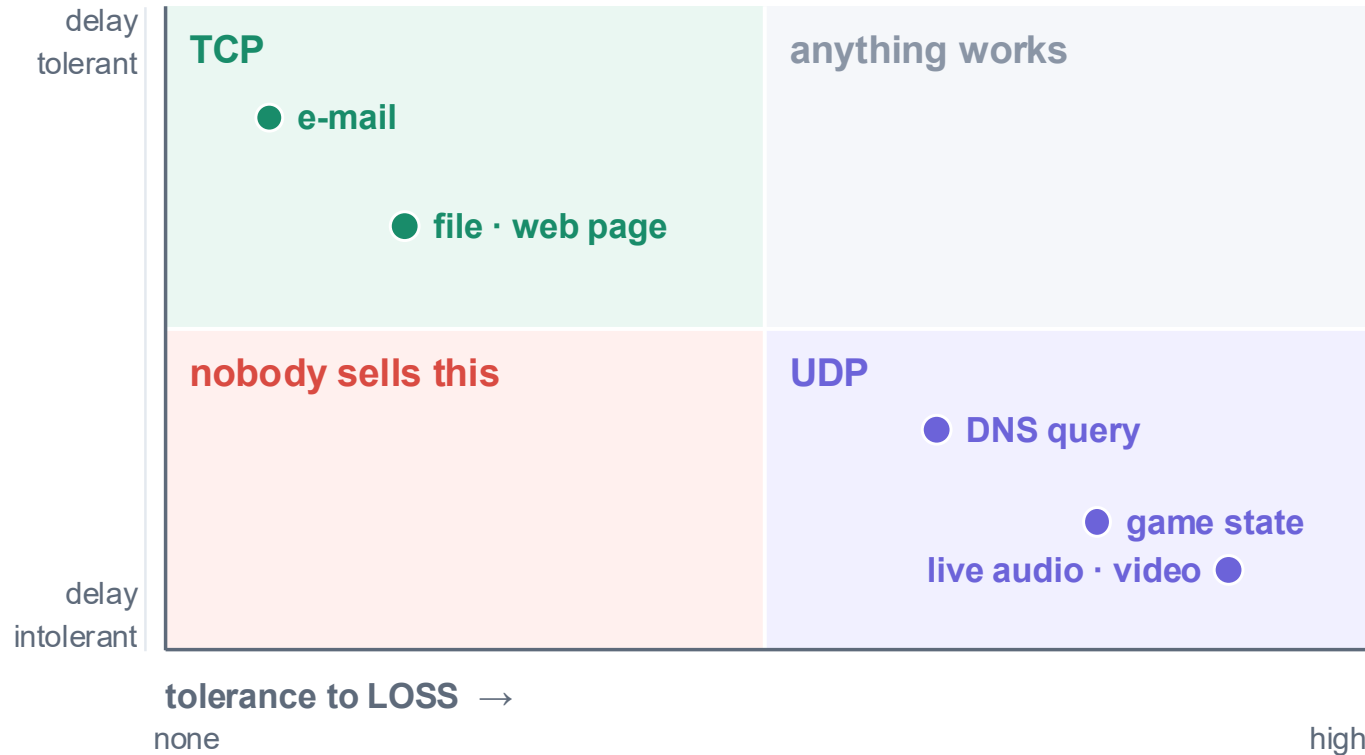
- **Where your messages begin and end**

TCP delivers a stream. It has no idea that you consider those 40 bytes to be one message rather than part of another.

**If you need to know the peer processed it, your protocol must say so. TCP never will.**

**Delivery is the transport's job. Correctness is still entirely yours.**

# Match the contract to the failure you cannot tolerate



## Top-left → TCP

Nothing may be missing and waiting is acceptable. Hand loss to the transport and never think about it again.

## Bottom-right → UDP

A gap is survivable; a late arrival is not. Keep the decisions in the application, where the deadline is known.

## Bottom-left → nothing

Lose nothing \*and\* be always on time is not a product the Internet sells. If you need it, you buy a private line.

**Choose from the failure you cannot tolerate—not from a feature comparison table.**

# Two things the transport layer will not give you

## No promise about performance

- **No minimum bandwidth**

There is no call that asks for 2 Mbps and is told yes. You measure what you are getting and adapt to it.

- **No maximum delay**

Delay depends on queues you cannot see, on a path you did not choose, and it changes from second to second.

- **No promise a path exists**

Routes change and links fail. TCP will eventually report a broken connection; it cannot prevent one.

## And no security at all

- **Anyone on the path can read it**

IP, TCP and UDP carry your bytes in the clear. Confidentiality was never part of the service.

- **Anyone can alter or inject**

Checksums catch accidents, not adversaries. Integrity is not part of the service either.

- **You do not know who the peer is**

An address says where the packet went, not who answered. Authentication is a separate problem with a separate solution.

**TLS exists because these are not transport guarantees—it adds them above transport, in your stack.**

**The gaps in the service are the specification for the work you still have to do.**

# How does your code hand bytes to the network?

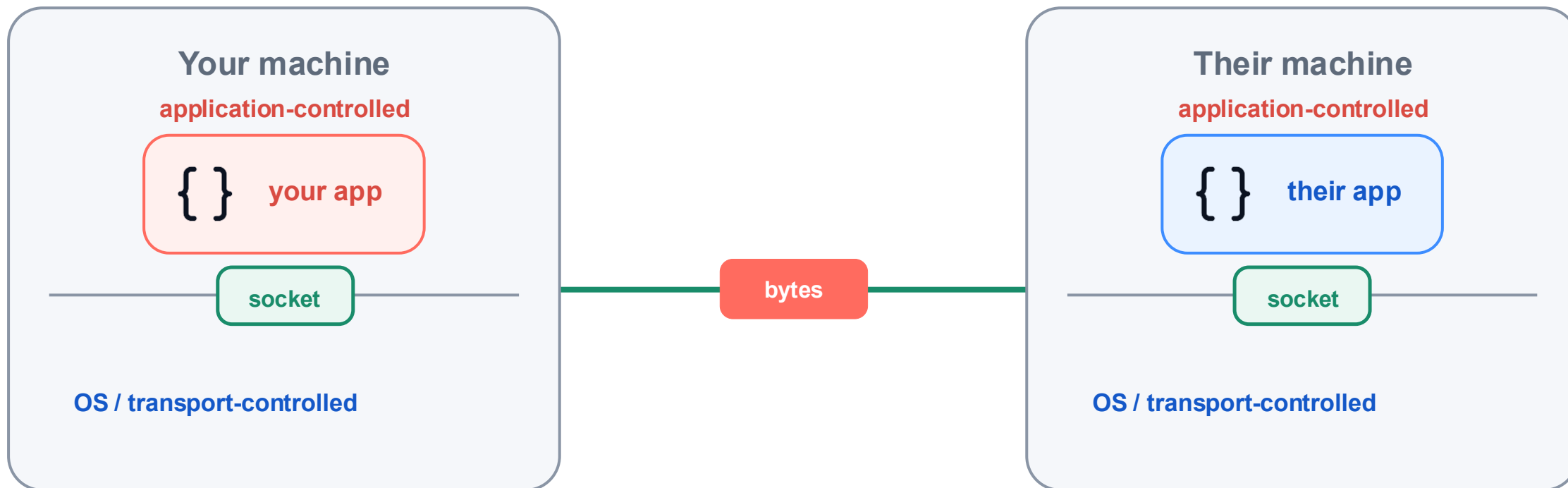
## 04

### Sockets and ports

A service you cannot reach is not a service. The interface is the line where your program stops and the operating system starts.

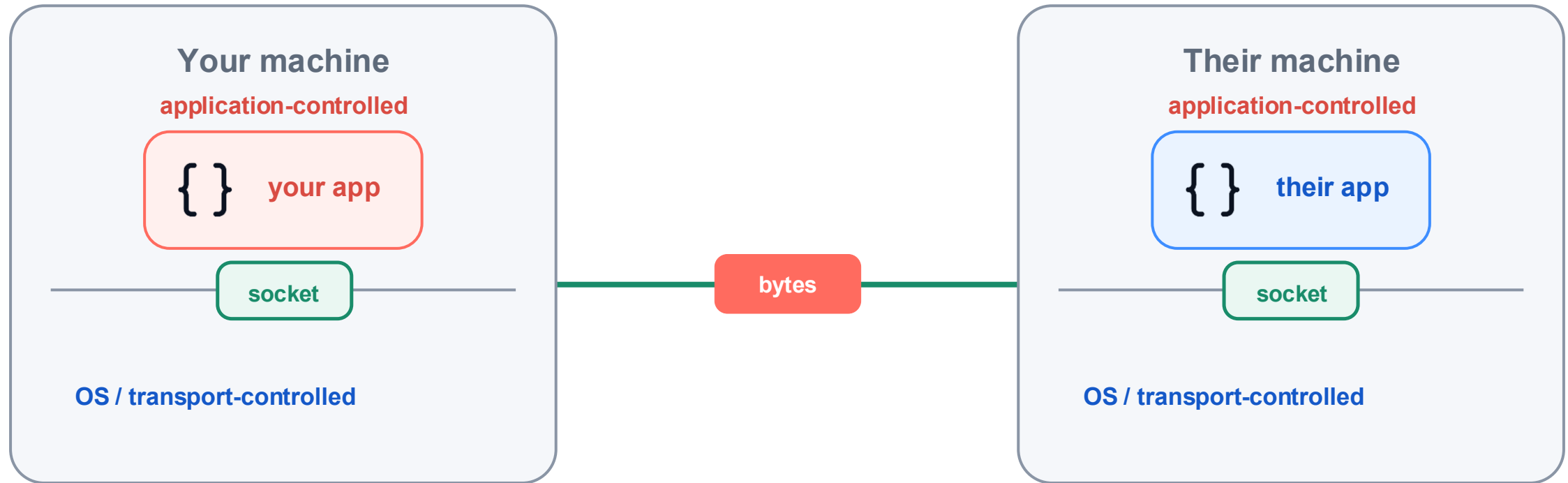
# Your program writes bytes in and reads bytes out

It never sees a packet, a header, a checksum, a timer, or a retransmission.



Everything below the socket is the operating system's problem—on both machines.

# The socket is the line where your responsibility ends



---

## Above the line — your program decides

- which bytes exist at all, and what they mean
- when to write them, and when to stop
- how a message is framed so the peer can find its edges

---

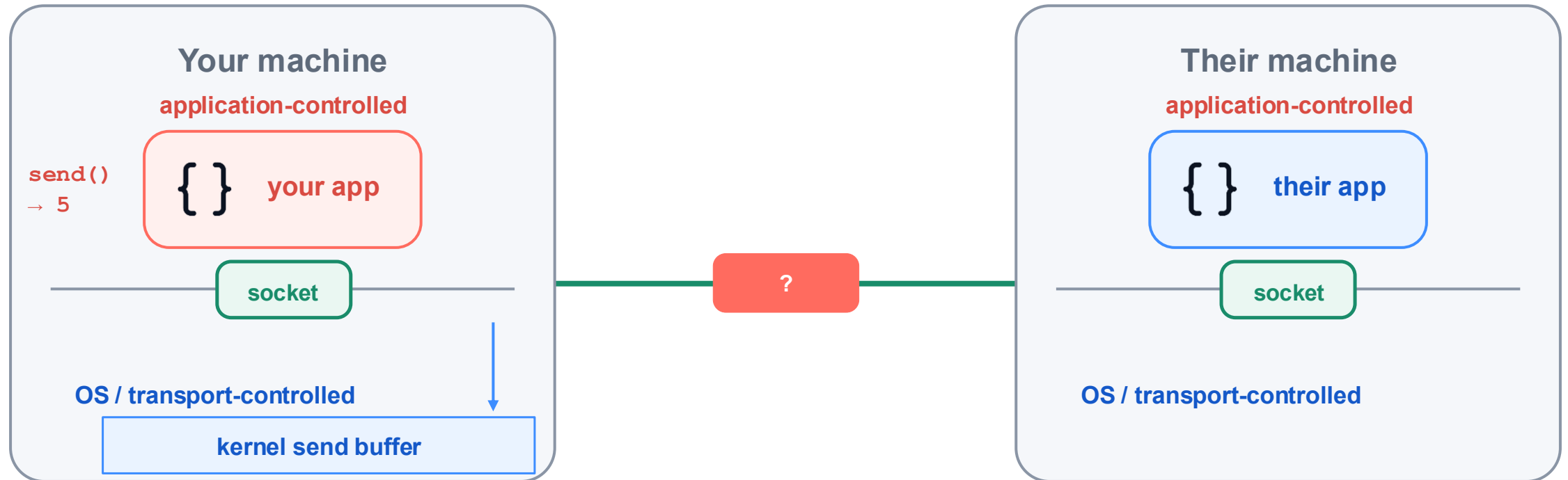
## Below the line — the OS decides

- how your bytes are cut into segments
- when each one is sent, resent, or held back
- which interface and which route it leaves by

- A misconception with real consequences

# send() returning success is a local event

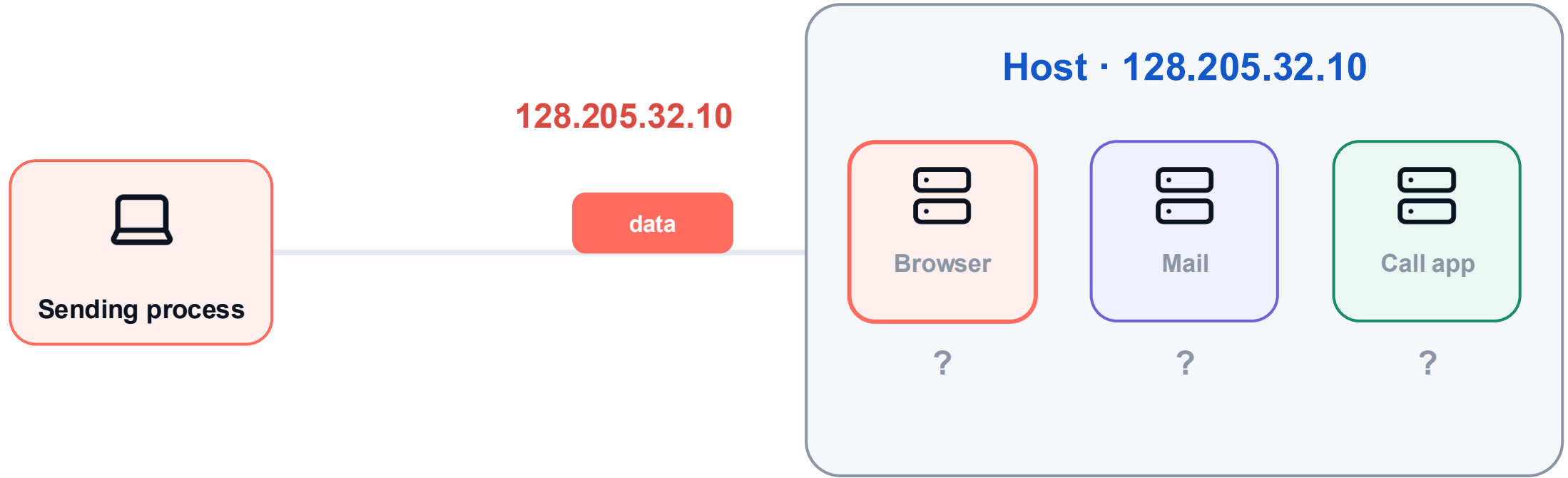
It tells you the kernel copied your bytes. It says nothing whatever about the peer.



At the instant `send()` returns, the peer may not have received the bytes; may not be running; may never have existed. All you have learned is that your process handed five bytes to your own operating system, and that it accepted them.

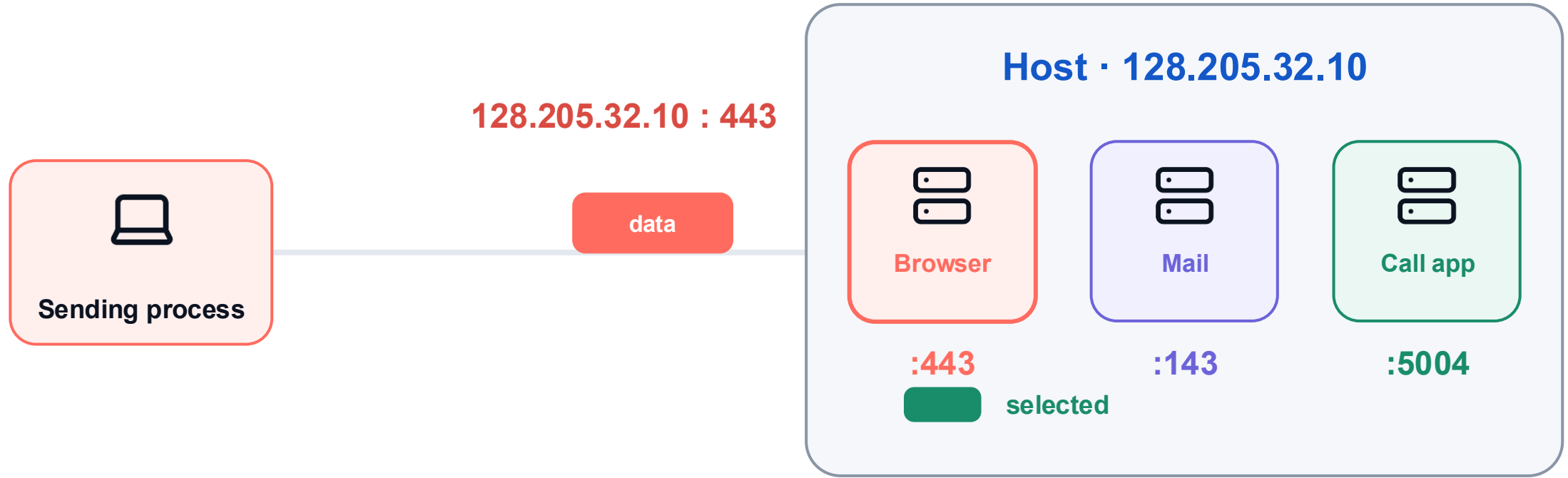
The only way to know the peer got it is for the peer to tell you — which means your protocol has to have a message that says so.

# The address gets the packet to the machine. Then what?



**A host may be running a hundred programs. The address does not choose between them.**

# The port number chooses which program receives it

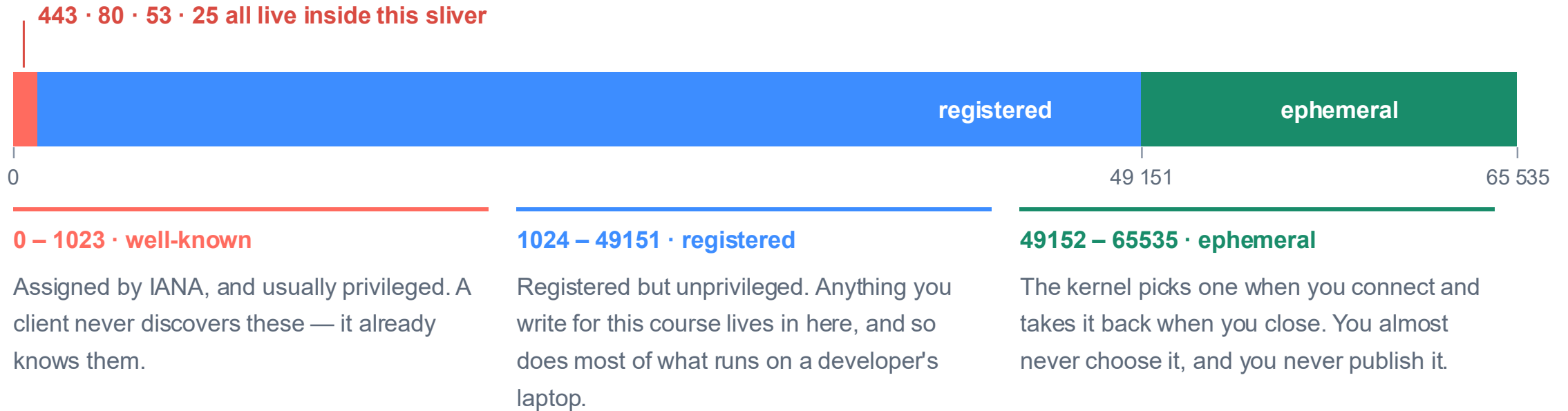


A port is not the program. It is a 16-bit number the kernel has bound to one socket — and the kernel owns that table, not you.

**The address selects a host; the port selects one transport endpoint on that host.**

# Only one side needs a port number fixed in advance

Every TCP or UDP port is a 16-bit number — 0 to 65535 — and they are not all yours.



The server needs a number the whole world already knows. The client's is borrowed for one connection and thrown away— which is why a laptop behind a NAT can use the Web perfectly well and cannot host it.

**Only the side that must be found in advance needs a fixed number.**

# One server port, thousands of simultaneous conversations

If everyone connects to :443, how does the server keep them apart?



Three connections, one server port. The operating system tells them apart by all four numbers:

|                        |   |                     |
|------------------------|---|---------------------|
| • 203.0.113.7 : 51514  | → | 128.205.32.10 : 443 |
| • 198.51.100.4 : 40912 | → | 128.205.32.10 : 443 |
| • 203.0.113.7 : 51515  | → | 128.205.32.10 : 443 |

Rows 1 and 3 are the same laptop. Only the source port differs — and that is enough to make them two separate connections.

**A TCP connection is named by four numbers. The server's port is only one of them.**

# One video call, all four questions answered

## 01 Roles

A meeting server is findable in advance; neither laptop is. It introduces the two peers, then gets out of the media path — unless a NAT forces it back in.

## 02 Contract

Signalling is request/response over TCP. Media is RTP: a protocol whose whole design assumes frames are timed, and that some of them will not arrive.

## 03 Delivery

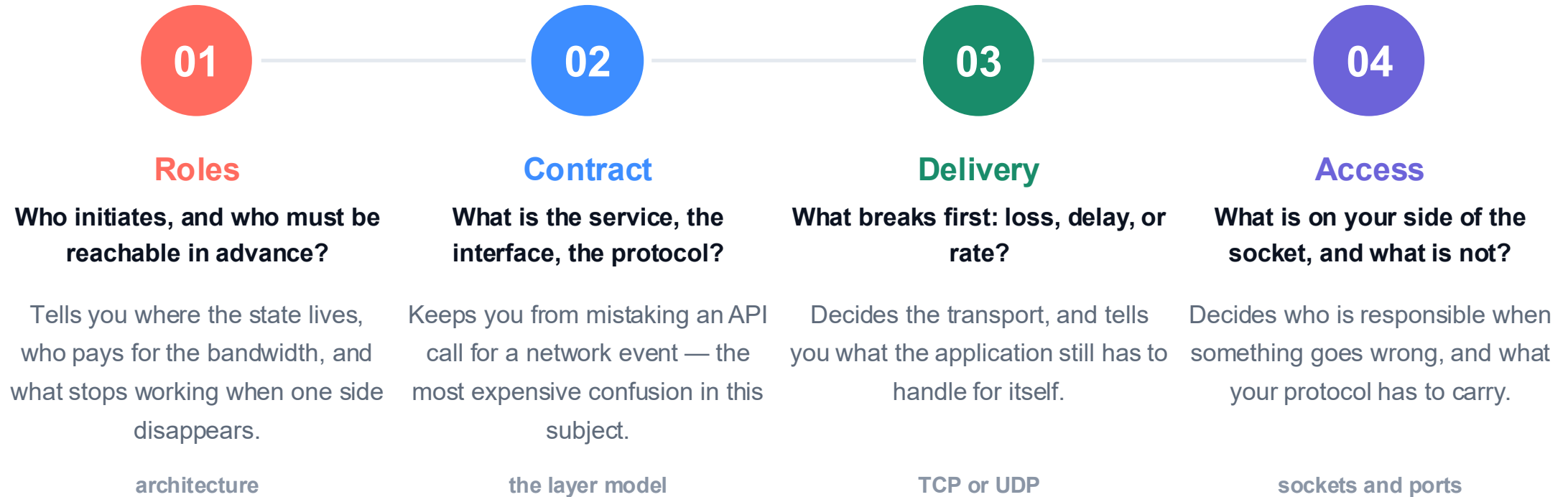
Audio and video take UDP, because late is worse than lost. The chat pane and the shared file take TCP, because nothing in them may be missing.

## 04 Access

Every flow is a socket. The server's signalling port is fixed and published; every other port in this picture was handed out by an operating system.

**Four decisions. Every application has made all four—whether or not anyone wrote them down.**

# Four questions to ask of any application you meet



**This method outlives every protocol in the syllabus—including the ones not yet invented.**

# The core stays ignorant. That is what leaves room for you.

Architecture decides who is findable. Transport offers a contract. The socket is where you reach it. The protocol makes the bytes mean something.

**Next: writing the sockets.**

CSE589 · Modern Network Concepts · Fall 2026

